Marathwada Mitra Mandal's

COLLEGE OF ENGINEERING, PUNE

An Autonomous Institute



Department of Computer Engineering

Lab Manual

410247 : LP-IV: Software Testing & Quality Assurance Lab

Prepared by,

Prof. Jagruti Wagh

BE COMP (2019 Pattern)

Academic Year 2026-27 Sem I

Preface

Software testing and quality assurance is a field of computer engineering focusing on the study, modeling, testing of computer applications. Software Quality is a critically important yet very challenging aspect of building complex software systems. This has been widely acknowledged as a fundamental issue for enterprise, web, and mobile software systems. However, assuring appropriate levels of software quality when building today's (and tomorrow's) adaptive, software-intensive and highly diverse systems is even more of a challenge.

There is a worldwide awareness of realizing the importance of software testing. software testing in the context of testing concepts and methods that can be implemented in practice. The main features of this subject is

- It focuses on the importance, significance and limitations of software testing.
- It presents functional testing techniques that may help to design test cases without considering the internal structure of the program.
- It presents structural testing techniques and introduces some fundamental structural testing techniques like manual and automation testing.
- It promotes verification testing as an integral to modern software testing practices, equally as important as validation testing. Verification activities such as walkthroughs and inspections can be carried out at early phases of software development.
- Use case testing has also been introduced. It addresses important issues of selection, minimization and prioritization of test cases for regression testing.
- Complementary approaches for achieving an adequate number of test cases during regression testing to show how to reduce maintenance cost.
- It presents software testing activities, debugging approaches and testing tools along with their commercial applications.
- It signifies the importance of object oriented testing. It also presents metrics used in software testing with their practical application in predicting models using commercial data sets.
- It emphasizes on testing web applications covering key areas such as functional testing, usability testing, user interface testing, security testing, performance testing, configuration and compatibility testing.
- It introduces the concepts of automated test data generation using static and dynamic test data generation techniques.


'येथे बहुतांचे हित ।'
Marathwada Mitra Mandal's
COLLEGE OF ENGINEERING
Karvenagar, Pune
Permanently affiliated to SPPU | Accredited with 'A' Grade by NAAC
Recipient of 'Best College' award in 2018-19 by SPPU
Vision
To aspire for the Welfare of Society
through excellence in Science and Technology.
Mission
Our Mission is to
Mould young talent for higher endeavours.
Meet the challenges of globalization.
Commit for social progress with values and ethics.
Orient faculty and students for research and development.
Emphasize excellence in all disciplines.

'येथे बहुतांचे हित ।'

Marathwada Mitramandal's
COLLEGE OF ENGINEERING

Karvenagar, Pune - 411052

Department of Computer Engineering

Vision

To contribute to welfare of society by empowering students with latest skills, tools and technologies in the field of Computer Engineering through excellence in education and research

Mission

- To provide excellent academic environment for continuous improvement in the domain knowledge of Computer Engineering to solve real world problems
- To impart value-based education to students, with innovative and research skills to make them responsible engineering professionals for societal upliftment
- To strengthen links with industries through partnerships and collaborative developmental works

Program Educational Objectives (PEOs)

- To develop globally competent graduates with strong fundamental knowledge and analytical capability in latest technological trends
- To prepare the graduates as ethical and committed professionals with a sense of societal and environmental responsibilities
- To inculcate research attitude in multidisciplinary domains with experiential learning and developing entrepreneurship skills
- To groom graduates by incorporating investigative approach among them to effectively deal with global challenges

Program Specific Outcomes (PSOs)

A graduate of the Computer Engineering Program will be able to

- Analyze the problems and design solutions in the areas of Artificial Intelligence & High Performance Computing
- Develop advanced digital solutions using standard software engineering practices

Program Outcomes (POs)

Engineering Graduates will be able to:

1. Engineering Knowledge: Apply the knowledge of mathematics, science, engineering fundamentals and an engineering specialization to the solution of complex engineering problems.
2. Problem Analysis: Identify, formulates, review research literature, and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.
3. Design / development of solutions: Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations.
4. Conduct investigations of complex problems: Use research – based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.
5. Modern tool usage: Create, select and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modeling to complex engineering activities with an understanding of the limitations.
6. The engineer and society: Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice.
7.Environment and sustainability: Understand the impact of the professional engineering solutions in societal and environmental contexts, and demonstrate the knowledge of, and need for sustainable development.
8. Ethics: Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.
9. Individual and team work: Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary settings.
10. Communication: Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.
11. Project management and finance: Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.
12. Life-long learning: Recognize the need for, and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.

Rubrics for Lab Assessment (CAS)

Dimension	Scale				
	1	2	3	4	5
Regularity and punctuality	Did not Perform, submitted in time	Performed and submitted later than scheduled date	Performed on schedule; submitted two weeks late	Performed on schedule; submitted one week late	Performed and submitted as per schedule
Understanding and preparation for Objective	Neither shows any understanding of the objective nor can relate it to theory.	States the objective very vaguely	Can only state the objective but shows poor understanding	Understands objective but cannot place it in context of a theory	Understands objective and can relate it to an appropriate theory topic
Participation in performance and conduction of experiment	Does not participate in experiment	Performs the experiment only with the help from supervisor/others and is confused and untidy.	Performs the experiment with some supervisory help; but forgets some crucial reading and is confused and untidy.	Performs experiment on own without supervisory help; records all readings properly but untidy.	Performs experiment on his/her own without supervisory help; records all readings properly. Keeps the setup clean and tidy.
Post experiment skills	Cannot follow the procedure and do any work	Follows procedure half-heartedly	Follows right procedure; but cannot analyze data and interpret it	Follows right procedure and can analyze data and interpret it	Follows right procedure; can analyze data and interpret it with justification

Syllabus

Savitribai Phule Pune University
Fourth Year of Computer Engineering(2019Course)
410247:Laboratory Practice IV

Teaching Scheme	Credit	Examination Scheme and Marks
Practical:04 Hours/Week	01	Term Work: 50 Marks

Companion Course: Elective III(410244), Elective IV(410245)

Course Objectives:

- Learn android application development related to pervasive computing
- Understand various multimedia file formats
- Understand various vulnerabilities and use of various tools for assessment of vulnerabilities
- Understand information retrieval process using standard tools available
- Learn GPU programming and implementation of same using open source libraries
- Learn installation and use of open source software testing tools

Course Outcomes:

After completion of the course, students will be able to
CO1: Apply android application development for solving real life problems
CO2: Design and develop system using various multimedia components.
CO3: Identify various vulnerabilities and demonstrate using various tools.
CO4: Apply information retrieval tools for natural language processing
CO5: Develop an application using open source GPU programming languages
CO6: Apply software testing tools to perform automated testing

Guidelines for Instructor's Manual

The instructor's manual is to be developed as a reference and hands-on resource. It should include prologue (about University/program/ institute/ department/foreword/ preface), curriculum of the course, conduction and Assessment guidelines, topics under consideration, concept, objectives, outcomes, set of typical applications/assignments/ guidelines, and references.

Guidelines for Student's Laboratory Journal

The laboratory assignments are to be submitted by student in the form of journal. Journal consists of Certificate, table of contents, and handwritten write-up of each assignment (Title, Date of Completion, Objectives, Problem Statement, Software and Hardware requirements, Assessment grade/marks and assessor's sign, Theory- Concept in brief, algorithm, flowchart, test cases, Test Data Set(if applicable), mathematical model (if applicable), conclusion/analysis. Program codes with sample output of all performed assignments are to be submitted as softcopy. As a conscious effort and little contribution towards Green IT and environment awareness, attaching printed papers as part of write-ups and program listing to journal must be avoided. Use of DVD containing students programs maintained by Laboratory In-charge is highly encouraged. For reference one or two journals may be maintained with program prints in the Laboratory.

Guidelines for Laboratory/Term Work Assessment

Continuous assessment of laboratory work should be based on overall performance of Laboratory assignments by a student. Each Laboratory assignment assessment will assign grade/marks based on parameters, such as timely completion, performance, innovation, efficient codes and punctuality.

Guidelines for Practical Examination

Problem statements must be decided jointly by the internal examiner and external examiner. During practical assessment, maximum weightage should be given to satisfactory implementation of the

problem statement. Relevant questions may be asked at the time of evaluation to test the student's understanding of the fundamentals, effective and efficient implementation. This will encourage, transparent evaluation and fair approach, and hence will not create any uncertainty or doubt in the minds of the students. So, adhering to these principles will consummate our team efforts to the promising start of student's academics.

Guidelines for Laboratory Conduction

The instructor is expected to frame the assignments by understanding the prerequisites, technological aspects, utility and recent trends related to the topic. The assignment framing policy need to address the average students and inclusive of an element to attract and promote the intelligent students. Use of open source software is encouraged. Based on the concepts learned. Instructor may also set one assignment or mini-project that is suitable to respective branch beyond the scope of syllabus.

Virtual Laboratory:
1. https://hci-iitg.vlabs.ac.in/
2. http://vlabs.iitkgp.ernet.in/se/
3. https://vlab.amrita.edu/?sub=3&brch=179&sim=1293&cnt=2

410245(D)Software Testing and Quality Assurance	
Any 5 assignments from group 1 and 1 Mini project from group 2 is mandatory	
Group 1:	
1.	Write TEST Scenario for Gmail Login Page
2.	TEST Scenario for Gmail Login Page
3.	Write Test cases in excel sheet for Social Media application or website
4.	Create Defect Report for Any application or web application
5.	Installation of Selenium grid and selenium Webdriver & java eclipse (automation tools).
6.	Prepare Software requirement specification for any project or problem statement
Group 2:	
7.	**Mini Project :**Software Testing and Quality Assurance Mini Project Dynamic webside of covid-19 information using HTML, CSS, JAVASCRIPT And PHP, MySQL database used to store user account, comment, and registration form details. Regular Expression testcases for testing purpose
8.	**Mini Project :**Create a small application by selecting relevant system environment / platform and programming languages. Narrate concise Test Plan consisting features to be tested and bug taxonomy. Prepare Test Cases inclusive of Test Procedures for identified Test Scenarios. Perform selective Black-box and White-box testing covering Unit and Integration test by using suitable Testing tools. Prepare Test Reports based on Test Pass/Fail Criteria and judge the acceptance of application developed
9.	**Mini Project :** Create a small web-based application by selecting relevant system environment / platform and programming languages. Narrate concise Test Plan consisting features to be tested and bug taxonomy. Narrate scripts in order to perform regression tests. Identify the bugs using Selenium WebDriver and IDE and generate test reports encompassing exploratory testing.

CO PO and PSO Mapping

A. Course Outcome

Course Outcome	Statement	BL
	At the end of the course, student will be able to	
410247.1	Identify various vulnerabilities and demonstrate using various tools.	BL1
410247.2	**Apply software testing tools to perform automated testings**	**BL4**

Course Outcome	Program Outcomes												PSO	
	1	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**	**10**	**11**	**12**	**1**	**2**
410247.1	2	1	1	2	-	-	-	-	3	1	-	-	-	2
410247.2	**2**	**1**	**2**	**2**	**1**	**-**	**-**	**-**	**3**	**1**	**-**	**-**	**-**	**2**

INDEX

Sr. No.	Ass No.	Title of Assignment	COs	POs
1	1	Write TEST Scenario for Gmail Login Page	2	1,2,3,4,5, 9,10
2	2	Write TEST Scenario for Gmail Inbox Functionality	2	1,2,3,4,5, 9,10
3	3	Write Test cases in excel sheet for Social Media application or website	2	1,2,3,4,5, 9,10
4	4	Create Defect Report for Any application or web application	2	1,2,3,4,5, 9,10
5	5	Prepare Software requirement specifications for any project or problem statement	2	1,2,3,4,5, 9,10
6	6	**Content Beyond Syllabus :**: Introduction of Junit Testing	2	1,2,3,4,5, 9,10
7	7	**Vlab :** Designing Test suite	2	1,2,3,4,5, 9,10
8		**Mini Project:** Software Testing and Quality Assurance Mini Project Dynamic website of covid-19 information using HTML, CSS, JAVASCRIPT And PHP, MySQL database used to store user account, comment, and registration form details. Regular Expression test cases for testing purpose **Or** **Mini Project** :Create a small application by selecting relevant system environments / platforms and programming languages. Narrate concise Test Plan consisting features to be tested and bug taxonomy. Prepare Test Cases inclusive of Test Procedures for identified Test Scenarios. Perform selective Black-box and White-box testing covering Unit and Integration test by using suitable Testing tools. Prepare Test Reports based on Test Pass/Fail Criteria and judge the acceptance of application developed **Or** **Mini Project** :Create a small web-based application by selecting relevant system environments / platforms and programming languages. Narrate concise Test Plan consisting features to be tested and bug taxonomy. Narrate scripts in order to perform regression tests. Identify the bugs using Selenium WebDriver and IDE and generate test reports encompassing exploratory testing.	2	1,2,3,4,5, 9,10

Use software testing concepts to implement the problem statements. Testing done using Selenium tools/Junit/pyunit.

Software Required:

1. 64 bit open source operating system
2. Geany
3. Selenium IDE/ webdriver
4. Eclipse version 3.8

Write-ups must include:

- **Assignment No.**
- **Title**
- **Problem Statement**
- **Prerequisites**
- **Course Objectives**
- **Course Outcomes**
- **Theory(in brief)**
- **Conclusion**
- **FAQs:**
- **Output: Printout of program with output.**

ASSIGNMENT NO: 1

TITLE: Test Scenario for Gmail Login Page.

PROBLEM STATEMENT: Write Test Scenario for Gmail Login Page

PREREQUISITES:

Understanding the basics concept of login functionality

COURSE OBJECTIVE:

Learn installation and use of open source software testing tools

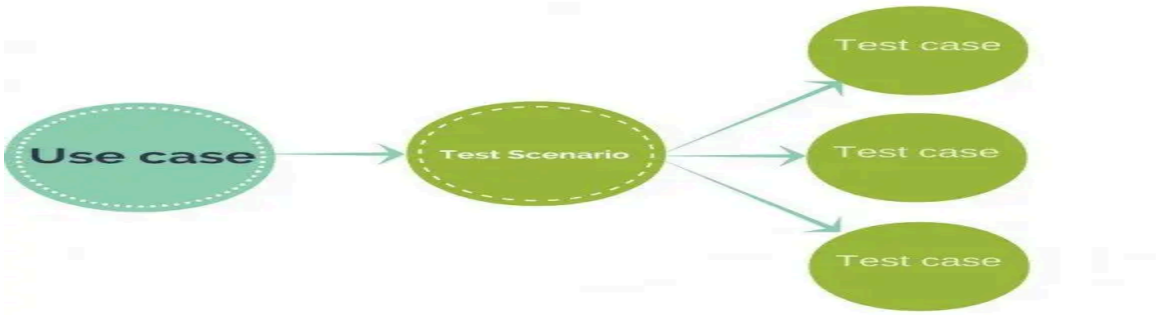
COURSE OUTCOME:

Apply software testing tools to perform automated testing

THEORY:

A Test Scenario is a statement describing the functionality of the application to be tested. It is used for end-to-end testing of a feature and is generally derived from the use cases.

Test scenarios can serve as the basis for lower-level test case creation. A single test scenario can cover one or more test cases. Therefore a test scenario has a one-to-many relationship with the test cases.



A Test scenario, sometimes also called a scenario test, is basically a documentation of a use case. In other words, it describes an action the user may undertake with a website or app. It may also describe a situation the user may find themselves in while using that software.

Test Scenarios are created to ensure that every single functionality offered by a website or app is working as expected. To create accurate test scenarios, it is best to gather input from clients, stakeholders, and developers. This helps effectively cover all possible user scenarios and enables

comprehensive testing of all business flows of the software in question.

Test Scenarios are required to verify the performance of the entire system from the users' perspective. When creating them, testers need to place themselves in the users' shoes so that they have clarity on what real-world scenarios the software will have to handle when made public.

Why create Test Scenarios?

- Helps ensure complete test coverage
- Since test scenarios must ideally be approved by business analysts, developers, and potential customers (a control group, ideally), they would really cover everything that needs to be tested before the software goes public
- Helps to estimate the testing effort required as well as organize the testing process. It can also serve as a proposal for the client
- Helps identify the most important end-to-end user flows, which assists with organizing the testing efforts according to priority

When to not use Test Scenarios

It is not necessary to create and use Test Scenarios in the following situations:

- In case of a time crunch, or when testing an unstable website or app. Creating detailed test scenarios takes time, and creating them for unstable software makes no sense because they will change as the software stabilizes.
- When a project is following the Agile methodology of development.
- When implementing a bug fix or regression tests. If this occurs, there will already be existing documentation from previous test cycles that can be used.

How to create a Test Scenario

- Study the Requirement Document – Business Requirement Specification (BRS), Software Requirement Specification (SRS), Functional Requirement Specification (FRS) pertaining to the System Under Test (SUT).
- Isolate every requirement, and identify what possible user actions need to be tested for it. Figure out the technical issues associated with the requirement. Also, remember to analyze and frame possible system abuse scenarios by evaluating the software with a hacker's eyes.
- Enumerate test scenarios that cover every possible feature of the software. Ensure that these scenarios cover every user flow and business flow involved in the operation of the website or app.
- After listing the test scenarios, create a Traceability Matrix to ensure that every

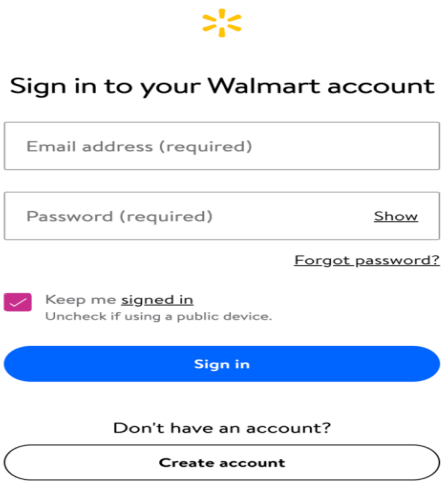
requirement is mapped to a test scenario.
Get the scenarios reviewed by a supervisor, and then push them to be reviewed by other stakeholders involved in the project.

Test Scenario Example

Let's have a look at an e-commerce website and a test scenario related to its Login page.

Test Scenario

Check the Login functionality.



Ask the following questions in this scenario:

- Check user log in with a valid email id and password
- Check system behavior when an invalid email id and valid password
- Check system behavior when a valid email id and invalid password is entered
- Check system behavior when an invalid email id and invalid password is entered
- Check system behavior when the email id and password fields are left blank and the Login button is clicked
- Check Forgot password option working as expected
- Check every link on the page working as expected?

Essentially, the test scenario should help testers check for every possible user action, and ensure that every action yields the expected result. After every test scenario is implemented, testers should be able to confidently say that they can answer every single question about that particular feature or function.

CONCLUSION: We have studied, Test scenario concept

FAQs:

Q.1 What is software testing

Q.2 What is Test Scenario

Q.3 Differentiate between Test Scenario and Test Case

Q.4 What Test suite?

Q.5 What is Test Condition?

OUTPUT

ASSIGNMENT NO: 2

TITLE: Test scenario for Gmail inbox functionality

PROBLEM STATEMENT: Write Test scenario for Gmail inbox functionality

PREREQUISITES:

Understanding the basics concept of test scenario

COURSE OBJECTIVE:

Learn installation and use of open source software testing tools

COURSE OUTCOME:

Apply software testing tools to perform automated testing

THEORY:

What is a Test Scenario?

A Test Scenario is defined as any functionality that can be tested. It is also called *Test Condition* or *Test Possibility*. As a tester, you should put yourself in the end user's shoes and figure out the real-world scenarios and use cases of the Application Under Test.

Scenario Testing

Scenario Testing in software testing is a method in which actual scenarios are used for testing the software application instead of test cases. The purpose of scenario testing is to test end to end scenarios for a specific complex problem of the software. Scenarios help in an easier way to test and evaluate end to end complicated problems.

- Each Test Scenario should be tied to a minimum of one Requirement or User Story as per the Project Methodology.
- Before creating a Test Scenario that verifies multiple Requirements at once, ensure you have a Test Scenario that checks that requirement in isolation.
- Avoid creating overly complicated Test Scenarios spanning multiple Requirements.
- The number of scenarios may be large, and it is expensive to run them all. Based on customer priorities only run selected Test Scenarios

CONCLUSION: We have write test scenarios for inbox functionality

FAQs:

Q. 1 What is meant by test coverage?

Q.2 What are the two main categories of software testing?

Q.3 What's the difference between verification and validation in testing?

Q.4 What are functional test cases and non-functional test cases?

Q.5 What are non-functional test cases

Output

ASSIGNMENT NO: 3

TITLE: Test cases for Social Media application or website

PROBLEM STATEMENT: Write Test cases in excel sheet for Social Media application or website

PREREQUISITES:

Understanding the basics concept of test case

COURSE OBJECTIVE:

Learn installation and use of open source software testing tools

COURSE OUTCOME:

Apply software testing tools to perform automated testing

THEORY:

A Test case refers to the sequence of actions required to verify a specific feature or functionality. Essentially, the test case details the steps, data, prerequisites, and postconditions necessary to verify a feature.

It will lay out particular variables which QAs need to compare expected and actual results to conclude if the feature is working as it should. Test case components mention input, execution, and expected output/response. Basically, it tells engineers what to do, how to do it, and what results are acceptable.

The Objective of Writing Test Cases

- To validate specific features and functions of the software.
- To guide testers through their day-to-day hands-on activity.
- To record a catalog of steps undertaken, which can be revisited in the event of a bug popping up.
- To provide a blueprint for future projects and testers so that they don't have to start work from scratch.
- To help detect usability issues and design gaps early on.
- To help new testers and devs quickly pick up testing, even if they join in the middle of an ongoing project.

Standard Test Case Format

- Test Case ID
- Test Scenario
- Test Steps
- Prerequisites
- Test Data
- Expected/Intended Results
- Actual Results
- Test Status – Pass/Fail

While writing test cases, remember to include:

- A reasonable description of the requirement
- A description of the test process
- Details related to test setup: version of the software under test, data points, OS, hardware, security clearance, date, time, prerequisites, etc.
- Any related documents or attachments testers will require
- Alternative to prerequisites, if they exist

Common Features of Test Cases

- **Likely to be revised and updated regularly.** Software requirements can change, depending on changes in business priorities or customer preferences. If requirements change, test cases will have to be altered accordingly. The detection of bugs and debugging steps may also require test cases to be changed.
- **Likely to involve clustering**. Test cases in a single test scenario usually have to be run in a specific sequence or in a group. In this case, particular pre-requisites of one test case will apply to other test cases in the same sequence.
- **Likely to be interdependent.** Often, test cases can depend on each other. This is especially true for layered applications with multi-tier business logic.
- **Likely to be used by testers as well as developers.** Test cases are helpful for developers as well as testers. For example, when devs are fixing bugs, test cases can be pretty useful to replicate said bug.

Test Case Example

Let's build a test case example based on a specific scenario. Here is a sample case.

- **Test Case ID**: #BST001
- **Test Scenario**: To authenticate a successful user login on Gmail.com
- **Test Steps:**

    - The user navigates to Gmail.com.
    - In the 'email' field, the user enters a registered email address.
    - The user clicks the 'Next' button.
    - The user enters the registered password.
    - The user clicks 'Sign In.'
- **Prerequisites**: A registered Gmail ID with a unique username and password.
- **Browser**: Chrome v 86. Device: Samsung Galaxy Tab S7.
- **Test Data:** Legitimate username and password.
- **Expected/Intended Results:** Once username and password are entered, the web page redirects to the user's inbox, displaying and highlighting new emails at the top.
- **Actual Results:** As Expected
- **Test Status – Pass/Fail**: Pass

Best Practices for Writing Test Cases

- Prioritize clarity and transparency. Be clear, concise, and assertive in describing what the tester needs to do and what results they should ideally get.
- Focus on End-User requirements when writing sample test cases. Map test cases to reflect every aspect of the user journey. Use the Specifications Document and the Requirements Document to do so.
- Avoid repetition. If multiple tests can be executed with the same test case, use the Test Case ID to refer to the required test case.
- Keep Test Steps as minimal as possible. Ideally, keep it to 10-15 steps, if possible.
- Focus on achieving maximum test coverage. While 100% test coverage is rarely achievable, a high percentage can be attained with the right strategies.
- Create self-cleaning test cases. That means test cases must revert the Test Environment to a pristine, pre-test state. Tests should not leave any remnants of themselves in the environment when they are completed. This is an integral element of Configuration Management. To understand more in depth**:** What is Configuration Management in DevOps?
- Shape test cases for tests that return the same results no matter who runs them. Ensure that tests are self-standing.

Once test cases have been shaped, corresponding tests must be run on real browsers, devices, and operating systems. Remember that device fragmentation is a significant concern for every developer and tester. Every website has to work seamlessly on multiple device-browser-OS combinations.

CONCLUSION: Students are able to prepare Test cases for any application.

FAQs:

Q.1 What is a test case?

Q.2 Who prepared the Test Case?

Q.3 What is a positive Test case and Negative test case?

Q.4 How do we derive Test cases?

Q.5 What is the difference between a use case and a test case?

OUTPUT

ASSIGNMENT NO: 4

TITLE: Defect report for website/application

PROBLEM STATEMENT: Create Defect Report for Any application or web application

PREREQUISITES:

Understanding the basics concept of defect and failure

COURSE OBJECTIVE:

Learn installation and use of open source software testing tools

COURSE OUTCOME:

Apply software testing tools to perform automated testing

THEORY:

Defect:

A defect in a software product is also known as a bug, error or fault which makes the software produce an unexpected result as per the software requirements. For example; incorrect data, system hangs, unexpected errors, missing or incorrect requirements.

Defect Report:

A defect report is a document that has concise details about what defects are identified, what action steps make the defects show up, and what are the expected results instead of the application showing error (defect) while taking particular step by step actions.

Defect reports are usually created by the Quality Assurance team and also by the end-users (customers). Often customers detect more defects and report them to the support team of the software development since the majority of the customers curiously tries out every feature in the application.

A typical defect report contains the information in an xls Sheet as follows.

- **Defect ID –** Every bug or defect has it's unique identification number
- **Defect Description –** This includes the abstract of the issue.
- **Product Version –** This includes the product version of the application in which the defect is found.
- **Detail Steps –** This includes the detailed steps of the issue with the screenshots attached

so that developers can recreate it.

- **Date Raised –** This includes the Date when the bug is reported
- **Reported By –** This includes the details of the tester who reported the bug like Name and ID
- **Status –** This field includes the Status of the defect like New, Assigned, Open, Retest, Verification, Closed, Failed, Deferred, etc.
- **Fixed by –** This field includes the details of the developer who fixed it like Name and ID
- **Date Closed –** This includes the Date when the bug is closed
- **Severity –** Based on the severity (Critical, Major or Minor) it tells us about impact of the defect or bug in the software application
    1. Low –
       A small bug that needs to be fixed and again it's not going to affect the performance of the software.
    2. Medium –
       This bug does affect the performance. Such as being an obstacle to do a certain action. Yet there is another way to do the same thing.
    3. High –
       It highly impacts the software though there is a way around to successfully do what the bug ceases to do.
    4. Critical –
       These bugs heavily impact the performance of the application. Like crashing the system, freezing the system or requiring the system to restart for work properly.
- **Priority –** Based on the Priority set (High/Medium/Low) the order of fixing the defect can be made. (Know more about Severity and Priority)
- **Attachments :** A sequence of screenshots of performing the step by step actions and getting the unexpected result. One can also attach a short screen recording of performing the steps and encountering defects. Short videos help developers and/or QA to understand the bugs easily and quickly.
- **Additional information :** The platform you used, operating system and version. And other information which describes the defects in detail for assisting the developer understand the problem and fixing the code for getting desired results.

Defect/ Bug report sample 1: Web Project bug report

Summary: In CTR (Click through ratio) 'Total' row calculation is wrong

Product: Example product

Version: 1.0

Platform: PC

URL: (Provide url of page where bug occurs)

OS/Version: Windows 2000

Status: NEW

Severity: Major

Priority: P1

Component: Publisher stats

Assigned To: developer@example.com

Reported By: tester@example.com

CC: manager@example.com

Bug Description:

Reproduce steps:

1) Go to page: (Provide URL of page where bug occurs)

2) Click on 'Publisher stats' link to view publisher's revenue detail stats date wise.

3) On page (Provide URL of page where bug occurs) check CTR value in 'Total' row of CTR stats table.

Actual result: Calculation of 'Total' row in CTR table is wrong. Also Individual row CTR for each publisher is not truncated to 2 digits after decimal point. It's showing CTR like 0.042556767.

Expected result: Total CTR= (Total clicks/Total searches)*100

[Attach bug screenshot if any]

Please fix the bug.

CONCLUSION: Thus, we have learnt to prepare defect report for any application or website

FAQs:

Q.1 What is Defect in Software Testing?

Q.2 Explain about Defect Life Cycle?

Q.3 What is Defect Management?

Q.4 What is a Defect Report?

Q.5 What is the producible defect?

OUTPUT

ASSIGNMENT NO: 5

TITLE: To prepare Software requirement specification.

PROBLEM STATEMENT:

Prepare Software requirement specifications for any project or problem statement

PREREQUISITES:

Understanding the basics of software engineering.

COURSE OBJECTIVE:

Learn installation and use of open source software testing tools

COURSE OUTCOME:

Apply software testing tools to perform automated testing

THEORY:

1. Introduction

1.1 Purpose

The purpose of this document is to present a detailed description of the project application. It will explain the purpose and features of the application, the interfaces of the software, what the software will do and the constraints under which it must operate. This document is intended for users of the application and also potential developers.

1.2 Product Scope

Provide a short description of the software being specified and its purpose, including relevant benefits, objectives, and goals. Relate the software to corporate goals or business strategies. If a separate vision and scope document is available, refer to it rather than duplicating its contents here

1.3 Overview

Describe the different types of reader that the document is intended for, such as developers, project managers, marketing staff, users, testers, and documentation writers. Describe what the rest of this SRS contains and how it is organized. Suggest a sequence for reading the document, beginning with the overview sections and proceeding through the sections that are most pertinent to each reader type.

1.4 References

List any other documents or Web addresses to which this SRS refers. These may include user interface style guides, contracts, standards, system requirements specifications, use case documents, or a vision and scope document. Provide enough information so that the reader could access a copy of each reference, including title, author, version number, date, and source or location.

2. Overall Description

2.1 Product Perspective

Describe the context and origin of the product being specified in this SRS. For example, state whether this product is a follow-on member of a product family, a replacement for certain existing systems, or a new, self-contained product. If the SRS defines a component of a larger system, relate the requirements of the larger system to the functionality of this software and identify interfaces between the two. A simple diagram that shows the major components of the overall system, subsystem interconnections, and external interfaces can be helpful.

2.2 Product Functions

Summarize the major functions the product must perform or must let the user perform. Details will be provided in Section 3, so only a high level summary (such as a bullet list) is needed here. Organize the functions to make them understandable to any reader of the SRS. A picture of the major groups of related requirements and how they relate, such as a top level data flow diagram or object class diagram, is often effective.

2.3 User Classes and Characteristics

Identify the various user classes that you anticipate will use this product. User classes may be differentiated based on frequency of use, subset of product functions used, technical

expertise, security or privilege levels, educational level, or experience. Describe the pertinent characteristics of each user class. Certain requirements may pertain only to certain user classes. Distinguish the most important user classes for this product from those who are less important to satisfy.

2.4 Operating Environment

Describe the environment in which the software will operate, including the hardware platform, operating system and versions, and any other software components or applications with which it must peacefully coexist.

2.5 Design and Implementation Constraints

Describe any items or issues that will limit the options available to the developers. These might include: corporate or regulatory policies; hardware limitations (timing requirements, memory requirements); interfaces to other applications; specific technologies, tools, and databases to be used; parallel operations; language requirements; communications protocols; security considerations; design conventions or programming standards (for example, if the customer's organization will be responsible for maintaining the delivered software).

3. External Interface Requirements

3.1 User Interfaces

Describe the logical characteristics of each interface between the software product and the users. This may include sample screen images, any GUI standards or product family style guides that are to be followed, screen layout constraints, standard buttons and functions (e.g., help) that will appear on every screen, keyboard shortcuts, error message display standards, and so on. Define the software components for which a user interface is needed. Details of the user interface design should be documented in a separate user interface specification.

3.2 Hardware Interfaces

Describe the logical and physical characteristics of each interface between the software product and the hardware components of the system. This may include the supported device types, the nature of the data and control interactions between the software and the hardware, and communication protocols to be used.

3.3 Software Interfaces

Describe the connections between this product and other specific software components (name and version), including databases, operating systems, tools, libraries, and integrated commercial components. Identify the data items or messages coming into the system and going out and describe the purpose of each. Describe the services needed and the nature of communications. Refer to documents that describe detailed application programming interface protocols. Identify data that will be shared across software components. If the data sharing mechanism must be implemented in a specific way (for example, use of a global data area in a multitasking operating system), specify this as an implementation constraint.

3.4 Communications Interfaces

Describe the requirements associated with any communications functions required by this product, including e-mail, web browser, network server communications protocols, electronic forms, and so on. Define any pertinent message formatting. Identify any communication standards that will be used, such as FTP or HTTP. Specify any communication security or encryption issues, data transfer rates, and synchronization mechanisms.

CONCLUSION: Students are able to prepare SRS for any project/ problem statement

FAQs:

Q 1. What Is A Software Requirements Specification?
Q 2. What Is Srs In Project?
Q.3 What Is Requirement Specifications Of The System?

Q.4 What Are The Requirements Of Software?

Q.5 What Is The Requirement Gathering?

Output

ASSIGNMENT NO: 6

TITLE: Introduction to Junit Testing

PROBLEM STATEMENT: Study features and working of Junit.

PREREQUISITES:

Understanding the basics of software engineering.

COURSE OBJECTIVE:

Learn installation and use of open source software testing tools

COURSE OUTCOME:

Apply software testing tools to perform automated testing

THEORY:

JUnit is a unit testing framework for the Java programming language. JUnit has been important in the development of test-driven development, and is one of a family of unit testing frameworks collectively known as xUnit, that originated with Junit.

Testing is the process of checking the functionality of an application to ensure it runs as per requirements. Unit testing comes into picture at the developers' level; it is the testing of single entity (class or method). Unit testing plays a critical role in helping a software company deliver quality products to its customers.

`Unit testing can be done in two ways – manual testing and automated testing.`

Junit is a unit testing framework for Java Programming Language. It plays a critical role in test-driven development, and is a family of unit testing frameworks collectively known as xUnit.

JUnit promotes the idea of "first testing then coding", which emphasizes on setting up the test data for a piece of code that can be tested first and then implemented. This approach is like "test a little, code a little, test a little, code a little." It increases the productivity of the programmer and the stability of program code, which in turn reduces the stress on the programmer and the time spent on debugging.

Features of JUnit

- JUnit is an open source framework, which is used for writing and running tests.
- Provides annotations to identify test methods.
- Provides assertions for testing expected results.
- Provides test runners for running tests.
- JUnit tests allow you to write codes faster, which increases quality.
- JUnit is elegantly simple. It is less complex and takes less time.
- JUnit tests can be run automatically and they check their own results and provide immediate feedback. There's no need to manually comb through a report of test results.
- JUnit tests can be organized into test suites containing test cases and even other test suites.
- JUnit shows test progress in a bar that is green if the test is running smoothly, and it turns red when a test fails.

What is a Unit Test Case ?

A Unit Test Case is a part of code, which ensures that another part of code (method) works as expected. To achieve the desired results quickly, a test framework is required. JUnit is a perfect unit test framework for Java programming language.

A formal written unit test case is characterized by a known input and an expected output, which is worked out before the test is executed. The known input should test a precondition and the expected output should test a post-condition.

There must be at least two unit test cases for each requirement – one positive test and one negative test. If a requirement has sub-requirements, each sub-requirement must have at least two test cases as positive and negative.

Sample JUnit Class:

```
public class Addition {
    public int add(int a,int b){
        return (a+b);
    }
}
```

Junit Testcase

```
package net.javaVillage;
import static org.junit.Assert.assertEquals;
```

```
import org.junit.After;

import org.junit.Before;

import org.junit.Test;
public class AdditionTest

        { private Addition addition;
        /** * Initialization */

        @Before

        public void setUp() {

                addition = new Addition();

        }


        /** * Test case for add method */

        @Test

        public void test() {

                int i = addition.add(3, 7);

                assertEquals(10, i);

        }


        /** * destroy the object */

        @After

        public void tearDown() {

                addition = null;

        }

}
```

CONCLUSION: We have studied, Junit testing concept

FAQs:

Q1. What is JUnit?

Q2. What is Unit Testing?

Q3.What are the features of JUnit?

Q4. What will happen if the return type of the JUnit method is String?

Q.5. What is the importance of @Test annotation?

ASSIGNMENT NO: 8

AIM: Virtual Lab Assignment (Designing Test Suite).

PROBLEM STATEMENT: To learn the Test suite

Theory: Test case preparation could begin right after the requirements identification stage. It is desirable (and advisable) to create a Requirements Traceability Matrix (RTM) showing a mapping from individual requirement to test case(s). A simplified form of the RTM is shown in table 1 (the numbers shown in this table are arbitrary; not specific to LIS).

Table 1: A simplified mapping from requirements to test cases

Requirement #	Test Case #
R1	TC1
R2	TC2, TC3, TC4
R3	TC5
R4	TC6

Table 1 states which test case should help us to verify that a specified feature has been implemented and working correctly. For instance, if test case # TC6 fails, that would indicate requirement # R4 has not fully been realized yet. Note that it is possible that a particular requirement might need multiple test cases to verify whether it has been implemented correctly.

To be specific to our problem, let us see how we can design test cases to verify the "User Login" feature. The simplest scenario is when both username and password have been typed in correctly. The outcome will be that the user could then avail all features of LIS. However, there could be multiple unsuccessful conditions:

- User ID is wrong
- Password is wrong
- User ID & password are wrong
- Wrong password given twice consecutively
- Wrong password given thrice consecutively
- Wrong password given thrice consecutively, and security question answered correctly
- Wrong password given thrice consecutively, and security question answered incorrectly

Table 2 shows the details of this test suite.

Table 2: A test suite to verify the "User Login" feature

#	TS1
Title	Verify "User Login" functionality
Description	To test the different scenarios that might arise while an user is trying to login

#	Summary	Dependency	Pre-condition	Post-condition	Execution Steps	Expected Output
TC1	Verify that user already registered with the LIS is able to login with correct user ID and password		Employee ID *149405* is a registered user of LIS; user's password is *this_is_password*	User is logged in	1. Type in employee ID as *149405* 2. Type in password *this_is_password* 3. Click on the 'Login' button	"Home" page for the user is displayed
TC2	Verify that an unregistered user of LIS is unable to login		Employee ID *149405xx* is not a registered user of LIS	User is not logged in	1. Type in employee ID as *149405xx* 2. Type in password *whatever* 3. Click on the 'Login' button	The "Login" dialog is shown with a *"Login failed! Check your user ID and password"* message
TC3	Verify that user already registered with the LIS is unable to login with incorrect password		Employee ID *149405* is a registered user of LIS; user's password is *this_is_password*	User is not logged in	1. Type in employee ID as *149405* 2. Type in password *whatever* 3. Click on the 'Login' button	The "Login" dialog is shown with a *"Login failed! Check your user ID and password"* message
TC4	Verify that user already registered with the LIS is unable to login with incorrect password given twice consecutively	TC3	This test case is executed after execution of TC3 before executing any other test case	User is not logged in	1. Type in employee ID as *149405* 2. Type in password *whatever2* 3. Click on the 'Login' button	The "Login" dialog is shown with a *"Login failed! Check your user ID and password"* message
TC5	Verify that user already registered with the LIS is unable to login with incorrect password given thrice consecutively	TC4	This test case is executed after execution of TC4 before executing any other test case	User is not logged in	1. Type in employee ID as *149405* 2. Type in password *whatever3* 3. Click on the 'Login' button	The "Login" dialog is shown with a *"Login failed! Check your user ID and password"* message; the security question and input box for the answer are displayed
TC6	Verify that a registered user can login after three consecutive failures by correctly answering the security question	TC5	This test case is executed after execution of TC6 before executing any other test case. Answer to the security question is *my_answer*.	Email sent containing new password. The email is expected to be received within 2 minute.	1. Type in the answer as *my_answer* 2. Click on the 'Email Password' button	Login dialog is displayed; an email containing the new password is received
TC7	Verify that a registered user's account is blocked after three consecutive failures and answering the security question incorrectly		Execute the test cases TC3, TC4, and TC5 once again (in order) before executing this test case	User account has been blocked	1. Type in the answer as *not_my_answer* 2. Click on the 'Email Password' button	The message *"Your account has been blocked! Please contact the administrator."* appears

In a similar way, test suites corresponding to other user requirements could be created as well. A good test plan can reduce the burden of the testing team by specifying what exactly they should focus on.

Conclusion: We have studied about test suite

Output: Screenshot of feedback